SMART CITY TRAFFIC MANAGEMENT SYSTEM

Advanced Data Structures Mini Project | C++ | Intermediate

Project report template — personalize the student, department, college, guide and academic-year fields before submission.

1. Abstract

The Smart City Traffic Management System is an educational prototype that models important city locations as vertices and roads as weighted edges in an undirected graph. The edge weight represents estimated travel time in minutes. The system applies Dijkstra's shortest-path algorithm with a min-priority queue to find a minimum-time route, Breadth-First Search (BFS) to identify reachable locations, and edge-weight updates to simulate congestion. A companion browser prototype demonstrates the same sample network interactively. The project uses static illustrative data and does not claim to provide live navigation or real-time traffic predictions.

2. Introduction

Urban mobility depends on road connectivity, travel time, traffic incidents and route selection. A graph is a natural abstraction: intersections or destinations become vertices, while roads become edges. Weighted shortest-path algorithms can support route-planning decisions when edge weights represent travel cost such as time or distance.

3. Problem Statement

Given a set of city locations connected by roads with positive estimated travel times, determine a route from a source to a destination whose total travel time is minimum. The system should also allow a road weight to be updated and report which locations are reachable from a chosen source.

4. Objectives

Represent a road network using an adjacency-list weighted graph.

Compute minimum estimated travel time using Dijkstra's algorithm.

Reconstruct and display the route using predecessor information.

Simulate congestion by updating a road's travel-time weight.

Use BFS to identify reachable locations.

Demonstrate the model through a simple interactive webpage.

5. Scope and Assumptions

The supplied network contains seven named locations and eleven undirected roads. Travel times are positive integer values and are illustrative. The model assumes the same travel time in both directions, no turn restrictions, no time-of-day variation and no live sensor feed. These assumptions keep the ADS concepts clear and testable.

6. Software and Hardware Requirements

Category	Requirement
Language	C++17
Compiler	GCC 9+ / Clang / MinGW-w64
Web prototype	Modern browser; HTML, CSS, JavaScript
Editor	VS Code or equivalent
Hardware	Standard laptop/desktop; 4 GB RAM recommended

7. Data Structures Used

Adjacency list: vector of vectors stores neighboring vertices and road weights efficiently.

unordered_map: maps a location name to its integer vertex index.

Min-priority queue: extracts the unsettled vertex with the smallest known distance.

Queue: supports BFS traversal.

Distance and parent arrays: store best-known distances and reconstruct the route.

8. Algorithm and Pseudocode

Dijkstra's algorithm (non-negative weights):

```
DIJKSTRA(G, source, destination)
   for each vertex v:
      dist[v] = INFINITY
      parent[v] = NIL
   dist[source] = 0
   insert (0, source) into min-priority-queue
   while queue is not empty:
```

```
    (d, u) = extract minimum
    if d != dist[u]: continue
    if u == destination: break
    for each edge (u, v, weight):
      candidate = dist[u] + weight
      if candidate < dist[v]:
        dist[v] = candidate
        parent[v] = u
        insert (dist[v], v)
if dist[destination] is INFINITY:
  report "No route"
else:
  follow parent[] from destination to source
  reverse and print path and dist[destination]
```

9. Flowchart

The editable Mermaid flowchart is supplied as docs/flowchart.mmd. Paste it into a Mermaid-compatible editor or GitHub Markdown to render it.

10. Complexity Analysis

Operation	Time complexity	Reason
Build adjacency list	O(V + E)	Insert vertices and road edges
Dijkstra	O((V + E) log V)	Adjacency list and binary min-heap
BFS	O(V + E)	Each vertex and edge is explored at most a constant number of times
Space	O(V + E)	Graph plus distance, parent, visited and queue structures

V = number of vertices (locations); E = number of edges (roads).

11. Implementation Modules

addLocation(): creates a vertex if it does not already exist.

addRoad(): inserts or updates an undirected weighted connection.

updateRoad(): changes both directions of an existing road.

shortestPath(): executes Dijkstra and reconstructs the selected route.

reachableFrom(): executes BFS and prints reachable locations.

displayRoads(): prints the adjacency-list representation.

12. Sample Demonstration

The initial sample network includes Central Station, City Hospital, Tech Park, Market Square, University, Airport and Bus Terminal. For example, the route Central Station → Tech Park → University → Airport has a total estimated time of 12 + 4 + 10 = 26 minutes. A route query should be executed in the program to verify the current result.

13. Testing

Test cases cover normal shortest-path queries, BFS connectivity, road-weight updates, unknown locations, invalid weights, invalid menu input and source-equals-destination. The full test matrix is provided in tests/test_cases.md.

14. Real-World Applications

Emergency response: route guidance for ambulances and fire services, when integrated with dispatch and current road conditions.

Logistics: selecting lower-time routes for delivery fleets.

Public transport planning: comparing travel-time connectivity between terminals and hubs.

Traffic operations: analyzing the impact of a congestion event or road closure on alternative routes.

Urban planning: identifying poorly connected areas and evaluating proposed road links.

15. Limitations

Static sample data; no GPS or live traffic API integration.

Undirected roads; one-way streets are not represented.

A single fixed weight cannot model time-dependent traffic.

No turn penalties, vehicle classes, road capacity or incident severity.

Shortest travel-time path is only as reliable as the supplied edge weights.

16. Future Enhancements

Use directed edges and represent one-way streets.

Integrate verified traffic feeds or simulated sensor updates.

Add A* search with a geographic heuristic.

Model time-dependent edge costs and road closures.

Display a map with geospatial coordinates and route overlays.

Add user roles, route history and analytics dashboard.

17. Conclusion

This project demonstrates how weighted graphs, adjacency lists, priority queues, Dijkstra's algorithm and BFS can solve a simplified urban route-planning problem. The C++ program provides the core ADS implementation, while the browser prototype makes the route and congestion concepts easier to demonstrate. A production traffic system would require accurate geospatial data, live traffic inputs, robust validation and operational integrations.

18. References

1. Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms, sections on graph algorithms.
2. C++ reference documentation: https://en.cppreference.com/
3. GCC documentation: https://gcc.gnu.org/onlinedocs/
4. Mermaid flowchart syntax: https://mermaid.js.org/syntax/flowchart.html